

Taylor-data identification of the double-series coefficients

Claude, with Daniel Murfet

2026-09-07

Abstract

Astra #10's R7. The Taylor-tree theorems take coefficient arrays $x, y: \mathbb{N}^2 \rightarrow \mathbb{R}$ as input; the paper writes them as Taylor coefficients of the phase and amplitude. We prove the identification: if $\xi(u, v) = \sum_{ij} x_{ij} u^i v^j$ on the open ball $|u|, |v| < \rho$ with $\|x\|_\rho < \infty$, then $\partial_v^j \partial_u^i \xi(0, 0) = i! j! x_{ij}$ (`taylorData_of_eq_dblSum`), so the coefficients are determined by the function on any ball (`dblSum_injective`). The one-variable engine is `iteratedDeriv_zero_of_hasSum`: $\partial^n f(0) = n! c_n$ for $f(y) = \sum c_n y^n$ on $|y| < r$ with $\sum |c_n| r^n < \infty$, a direct consequence of Mathlib's `HasFPowerSeriesOnBall.factorial_smul` applied to the scalar series `FormalMultilinearSeries.ofScalars`. Zero sorry/axiom.

1 The things formalised

```
theorem iteratedDeriv_zero_of_hasSum (c :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (r) (hr : 0 < r)
  (hs : Summable fun n => |c n| * r ^ n) (f :  $\mathbb{R} \rightarrow \mathbb{R}$ )
  (hf :  $\forall y, |y| < r \rightarrow \text{HasSum } (\text{fun } n \Rightarrow c n * y ^ n) (f y)$ ) (n) :
  iteratedDeriv n f 0 = n.factorial * c n
def rowNorm (x) (i) :  $\mathbb{R} := \sum_j |x (i, j)| * r ^ j$ 
theorem row_summable / rowNorm_mul_summable / abs_faceU_le_rowNorm / faceU_hasSum
theorem dblSum_hasSum_faceU (h) (x) (hx : WSummable x) (u v)
  (hu : |u| < r) (hv : |v| < r) :
  HasSum (fun i => faceU x i v * u ^ i) (dblSum x u v)
theorem taylorData_of_eq_dblSum (h) (x) (hx) (i j) :
  (h :  $\forall u v, |u| < r \rightarrow |v| < r \rightarrow u v = \text{dblSum } x u v$ ) (i j) :
  iteratedDeriv j (fun v => iteratedDeriv i (fun u => u v) 0) 0
  = i.factorial * j.factorial * x (i, j)
theorem dblSum_taylorData ... (x) (hx) (i j) :
  iteratedDeriv j (fun v => iteratedDeriv i (fun u => u v) 0) 0
  = i.factorial * j.factorial * x (i, j)
theorem dblSum_injective (h) (x x') (hx hx')
  (h :  $\forall u v, |u| < r \rightarrow |v| < r \rightarrow \text{dblSum } x u v = \text{dblSum } x' u v$ ) : x = x'
```

2 Mathematics

For a scalar power series $\sum c_n y^n$ with $\sum |c_n| r^n < \infty$, the formal multilinear series $p_n = c_n \cdot \text{mkPiAlgebraFin}$ has radius $\geq r$ and sums to f on the ball, so `HasFPowerSeriesOnBall` `f p 0 r`; Mathlib's `factorial_smul` then gives $n! p_n(1, \dots, 1) = D^n f(0)(1, \dots, 1) = \partial^n f(0)$, and $p_n(1, \dots, 1) = c_n$. Nothing about the function off the ball is used, which is what makes the lemma applicable to functions that only agree with the series near the origin. The two-variable statement is two applications: for fixed $|v| < \rho$ the function $u \mapsto \xi(u, v)$ is the series $\sum_i a_i(v) u^i$ with $a_i(v) = \sum_j x_{ij} v^j$ (`faceU`), absolutely convergent since $|a_i(v)| \leq \sum_j |x_{ij}| \rho^j$ and $\sum_i (\sum_j |x_{ij}| \rho^j) \rho^i = \|x\|_\rho$; so $\partial_u^i \xi(0, v) = i! a_i(v)$. Then $v \mapsto i! a_i(v)$ is the series with coefficients

$i!x_{ij}$, and one more application gives $i!j!x_{ij}$. Injectivity follows by applying the identification to x' against the function `dblSum x`.

3 Paper-facing meaning

This closes the gap between the paper’s phrasing (“ x_{ij} are the Taylor coefficients of ξ at the origin”) and the Lean input (an abstract array with $\|x\|_\rho < \infty$): any analytic ξ on the ρ -ball with such a representation has exactly these coefficients, and the canonical Taylor-tree coefficients A_α, B_α are therefore functions of the germ of (ξ, η) at the origin. With units 138–143 the whole chain $\text{germ} \mapsto \text{coefficient arrays} \mapsto \text{amplitude coefficients} \mapsto (A_\alpha, B_\alpha)$ is now formalised, the last step locally Lipschitz. Astra #10’s R7 is complete; what remains from that consult is the R9 expectation bridge for $\mathbb{E}[\log Z]$, which needs the §4.3 fluctuation-function definition first.

4 Lean notes

- `FormalMultilinearSeries.le_radius_of_summable_norm` and `ofScalars_norm` (needs `NormOneClass`, fine over $\mathbb{R}$) give the radius bound; `ofScalars_apply_eq` evaluates $p_n(y, \dots, y) = c_n \cdot y^n$; `iteratedDeriv_eq_iteratedFDeriv` is rfl and bridges to `factorial_smul`.
- Ball membership for an `NNReal` radius: `Metric.eball_coe` (`Metric.emetric_ball_nnreal` is deprecated on this pin), then `Metric.mem_ball`, `dist_zero_right`.
- `summable_prod_of_nonneg` with a `by positivity` nonnegativity argument fails when the term is presented as an unreduced lambda application; pass an explicit `mul_nonneg (abs_nonneg _) (pow_nonneg _ _)` term instead.
- The style linter flags goal-changing `show` (use `change`); the project-wide “Copyright too short!” header lint fires on every Grammar file and is not specific to this unit.
- Row-wise resummation is `HasSum.prod_fiberwise` applied to the absolutely convergent double family, with each row’s `HasSum` from `faceU_hasSum.mul_right`.